Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

CS101

Thinking before typing

Today
Welcome to your next lesson: Thinking before typing

I'm preparing your personalized lesson based on what you've learned, what's coming up, and the best way for you to build on your skills.

Step by step, you’re moving forward.

Imagine someone tells you:

“Build a Python tool that picks a random snack for me when I’m bored.” 😄

Most beginners open the editor and start coding right away. Then 20 minutes later they realize:

What counts as a “snack”?
Should it avoid repeats?
Can I give it my own list of snacks?

All those missing details = rework.

In this lesson, we’ll learn to slow down for a minute and turn that kind of request into a clear spec before typing code.

Quick check for you, Ron:

When you see a request like “Build a tool that picks a random snack for me,” what’s your first instinct right now?

A) Start writing Python code

B) Ask a few clarifying questions

C) Write a short spec (even messy) before coding

Just tell me A, B, or C — no explanation needed.

id say c though mostly visualiing and thinking more then journaling

Nice, C is exactly the habit we’re building here 👀

And “visualizing and thinking” totally counts — we’ll just make it a bit more structured.

Let’s start with one pillar of a spec: inputs.

Plain request:

“Build a Python tool that picks a random snack for me when I’m bored.”

If we only think about inputs, we ask:

“What information does the program need from the outside to do its job?”

For this snack tool, possible inputs might be:

A list of snacks the user likes
Maybe how many snacks to pick
Maybe a flag like “no repeats today”

We’re not coding any of this, just naming the pieces the tool needs to receive.

Does this description of inputs = what the program needs to receive make sense so far?

Yes, that makes sense

What do you think? Pick the option that best matches an input 👇

For the snack tool request:

"Build a Python tool that picks a random snack for me when I’m bored."

Which of these is an input?

A.

A list of snack names the user likes

B.

A rule that the snack must not be repeated in one day

C.

The printed message: "Here is your snack: chips"

D.

The fact that the program is written in Python

🎯 Yes, exactly right: “A list of snack names the user likes” is an input.

The program needs to receive that list from somewhere (user, file, hard□coded later) so it knows what it can choose from.
The “no repeats in one day” is more like a rule/constraint, and
The printed message is an output, not an input.
Next step, staying slow and visual 🧠✨:

Let’s add a second pillar: expected outputs.

Same request:

“Build a Python tool that picks a random snack for me when I’m bored.”

If we only think about outputs, we ask:

“After the program runs once, what exactly should come out — in what form?”

For example, possible outputs:

A single snack name (like "chips")
Or a full sentence like: "Here is your snack: chips"
Maybe also a log line, or nothing else at all

Again, no code — just describing what comes out.

Does outputs = what the program gives back or shows feel clear?

If yes, I’ll give you a tiny scenario to pick the output from.

Yes, outputs are clear

What do you think the output is here? Pick one option in the question box 👇

For the snack tool, which one is the expected output of a single run?

A.

The list of all snacks the user likes

B.

The string: "Here is your snack: chips"

C.

The rule: "Do not repeat snacks in one day"

D.

The number of snacks the user has in their list

Nice job again, Ron 🎯

“Here is your snack: chips” is the output because it’s what the user actually sees/gets back after one run.

Inputs = what the program needs to receive
Outputs = what the program returns or displays
Now we add a third pillar: constraints.

Same request:

“Build a Python tool that picks a random snack for me when I’m bored.”

Constraints are the rules the solution must follow, like:

“Do not repeat a snack in the same day”

"Only pick from snacks marked as 'healthy'"
"Must respond in under 1 second"

They're not new inputs or outputs — they're rules about how the program behaves.

Does constraints = rules the program must obey feel clear?

If yes, I'll give you a quick multiple□choice to spot a constraint next.

Yes, that's clear

What do you think counts as the constraint here? Take your time and pick one option 👇

For the snack tool, which of these is a constraint?

A.

The list of snacks the user likes

B.

The string shown: "Here is your snack: chips"

C.

The rule: "Do not repeat a snack in the same day"

D.

The fact that the program uses Python

Exactly right again 🎯

"Do not repeat a snack in the same day" is a constraint because it's a rule about how the program must behave, not something it receives or prints.

So far you've nailed:

Inputs → what the program needs to receive
Outputs → what the program returns/shows
Constraints → rules the program must obey
Last pillar: edge cases.

Same request:

"Build a Python tool that picks a random snack for me when I'm bored."

Edge cases are the weird or extreme situations the program still has to handle, like:

The user's snack list is empty
The user only has 1 snack, but we say "no repeats today"
The snack name is super long or has strange characters

They're not normal flow — they're the "what if things are a bit broken/odd?" situations.

Does edge cases = unusual or tricky situations the program must still handle make sense?

If yes, I'll give you one last question to spot an edge case for a username field.

yes

What do you think? Pick the option that best shows an edge case 👇

For a username input, which one is an edge case the program should still handle?

A.

A username that is 8 characters long with only letters

B.

A username that is empty ("")

C.

A username that uses only lowercase letters

D.

A username that clearly follows all normal rules

You're on a roll, Ron 👏

Yes — an empty username "" is an edge case, because it's not the "normal" situation but the program still has to decide what to do with it (reject it, show an error, etc.).

So now you've got all four pillars of a spec:

Inputs – what the program receives
Outputs – what it returns/shows
Constraints – rules it must obey
Edge cases – weird/tricky situations it still must handle
To keep this super practical, I'd like to walk through one full small example spec with you next, using a simple feature like:

"Make a Python function that checks if a number is even."

We'd fill in a tiny table: inputs, outputs, constraints, edge cases.

Are you up for doing that together step by step?

Yes, let's do it!

Awesome 🙌 Let's keep it very step-by-step and visual.

We'll use this plain request:

"Make a Python function that checks if a number is even."

For now, we only care about inputs.

Think:

"What does this function need to receive from the outside to do its job?"

A simple way to picture it:

Pillar	Content (for "is even" function)
Input	?
Output	
Constraints	
Edge cases	

For this function, I'd say the input is:

A single number (the value we want to check)

Nothing else is needed to decide if it's even.

Quick check:

For this "is even" function, do you agree the input is just one number to check, or do you feel like it needs more than that?

yes

Perfect ☑ One number is all it needs as input.

Let's extend the same table now to outputs.

"Make a Python function that checks if a number is even."

We ask:

"After the function runs once, what exactly should it give back?"

Two common choices:

It returns a boolean: True if even, False if odd
Or it returns a string like "Even" or "Odd"

Both can work, but a clean, typical spec might say:

Pillar	Content
Input	One number to check
Output	A boolean: True if even, False if odd

So the output is not "printing on screen" in the spec — it's what the function returns.

Does that idea of output = what the function returns after one call feel solid here?

Yes, that makes sense

Nice 👍 You've got inputs and outputs lined up.

Let's add the next pillar for the same request:

"Make a Python function that checks if a number is even."

Now we think about constraints = rules the function must obey.

Some possible constraints for this function:

It must treat 0 as even.
It must work for negative numbers too.
It must only accept integers (if a float like 3.5 is passed, that might be invalid).

So our table grows:

Pillar	Content
Input	One number to check
Output	A boolean: True if even, False if odd
Constraints	Only integers allowed; 0 is even; negatives handled as usual

We're still not coding, just writing the rules.

Before we add edge cases, quick check:

Does it feel clear why something like "0 is even" or "only integers allowed" belongs under constraints rather than inputs/outputs?

yers

Great ☑

Last pillar for this function: edge cases.

Request:

"Make a Python function that checks if a number is even."

Edge cases = unusual or tricky inputs the function must still handle.

Based on the constraints we mentioned, good edge cases might be:

0 (boundary, but valid even number)
A negative even number (like -4)
A negative odd number (like -3)
A non-integer, like 3.5 or "hello" (if someone calls it wrong)

So our little spec table looks like:

Pillar	Content
Input	One number to check
Output	Boolean: True if even, False if odd
Constraints	Only integers allowed; 0 is even; negatives handled as usual
Edge cases	0, negative numbers, non-integer input (e.g., 3.5, "hello")

This is a four□pillar spec: someone else could now implement is_even in Python without guessing.

I'd love to have you try one yourself next, using a different simple feature and we'll

build the table together.

Would you rather do a spec for:

A) A function that returns the maximum of two numbers

B) A function that checks if a password is strong enough

b

Nice choice 😄 Passwords are great for thinking about rules and edge cases.

Feature request:

"Make a Python function that checks if a password is strong enough."

We'll build the same 4□pillar table, one row at a time.

First pillar: Input

Ask: "What does this function need to receive from the outside?"

For this feature, the natural input is:

One password string to check

So we start:

Pillar	Content
Input	One password string to check
Output	
Constraints	
Edge cases	

Step 1:

Do you agree the input is just one password string, or would you add anything else as an input here?

yes

Perfect 👍 We'll keep it as:

Pillar	Content
Input	One password string to check

Next pillar: Output.

Request:

"Make a Python function that checks if a password is strong enough."

We ask:

"After one call, what should the function give back?"

Two common choices:

Return a boolean: True if strong, False if not
Or return a message string like "Strong password" / "Too short"

Let's pick one to make the spec precise. I suggest:

Output: a boolean – True if strong, False otherwise

So we'd have:

Pillar Content
Input   One password string to check
Output Boolean: True if strong, False otherwise
Do you like the boolean output choice here, or would you prefer it to return a message string instead?

sure

Got it 👍 We'll stick with the boolean output.

So our table so far:

Pillar Content
Input   One password string to check
Output Boolean: True if strong, False otherwise

Next pillar: Constraints = rules for what "strong enough" means.

Typical rules could be, for example:

Minimum length (say, at least 8 characters)
Must contain at least one digit
Must contain at least one uppercase letter

Then the spec line might look like:

Pillar Content
Constraints Length ≥ 8; at least 1 digit; at least 1 uppercase letter
Let's do this together:

If you had to pick just 2 rules for a "strong" password for now, which two would you choose? (plain English, no need for perfect wording)

miimum length 8 and must contain a digit ..

Great choices 👌 Those are clear and simple rules.

So our spec now:

Pillar Content
Input   One password string to check
Output Boolean: True if strong, False otherwise

Constraints Length ≥ 8; must contain at least one digit
Edge cases

Next, edge cases for this password check, based on your rules:

We look for “weird but possible” passwords, like:

Exactly 8 characters with no digit → should be False
Very long password but no digit at all → False
Password that is only digits, like "12345678" → is that allowed as strong under your rules? (It meets length and digit, but maybe too simple.)

These are edge cases you decide in the spec.

Let’s keep it simple:

Can you name 1 edge case password (just write an example string like "password" or "1234567") that our function should definitely treat as not strong under your two rules?

"" "letmein" "password"

Nice set of edge cases, Ron 👍

All three of these are good “not strong” examples under your rules:

"" → length 0, fails minimum length 8
"letmein" → length 7, also no digit
"password" → length 8, but no digit

Let’s plug one into the table so it’s explicit:

Pillar	Content
Input	One password string to check
Output	Boolean: True if strong, False otherwise
Constraints	Length ≥ 8; must contain at least one digit
Edge cases	"password" → length 8 but no digit, so result must be False

This is now a full four□pillar spec.

Someone else could implement this function in Python without guessing.

To check that this really “clicked”, let’s try one more tiny spec in a different domain, but even simpler: a function that returns the maximum of two numbers.

I’ll write a rough table with some blanks, and you fill them in:

Pillar	Content
Input	[your answer]
Output	[your answer]
Constraints	both values are numbers (e.g. ints or floats)
Edge cases	both numbers equal (e.g. 5 and 5)

Can you fill in:

What are the inputs?
What is the output?

Short phrases are enough, like “two numbers” or “the larger number”.

inpiut : a set of 2 numbers (no idea what a maximum of 2 numbers means however , the larger of the 2?)

output: assuming that, the maximum number
Conrtaints: must be 2 numbers not 0 or 1 or 3+, numbers must not be hthe same
Edge cases: Empty list passed, just 1 number pased, strings not numbers past

I like how you thought this through and even questioned the wording 👌

Let’s tighten it just a bit so it matches the idea of “max of two numbers” very clearly.

Step by step:

“Maximum of two numbers”

Yes, it just means: the larger of the two values.
If both are equal, that value is still the maximum.

A clean spec table could look like this:

Pillar	Content
Input	Two numbers (e.g. a and b)
Output	The larger of the two numbers; if they’re equal, return that value
Constraints	Both values are numbers (ints/floats); function takes exactly 2 inputs
Edge cases	a == b (e.g. 5 and 5); negative numbers; very large or very small

Notice what changed vs. your draft:

We allow numbers to be the same (that’s an edge case, not forbidden).
Edge cases match the function’s signature (we don’t talk about lists here; just two params).
You’re doing exactly what I want in this lesson: thinking before typing, and questioning the spec wording itself. 🔍

To lock in the four‑pillar idea, I want to do one quick multiple‑choice: I’ll show you four short statements about this “max of two numbers” function, and you’ll pick which one is an edge case.

Ready for that quick check? (just “yes” or “no”)

Yes

What do you think? Pick the option that best represents an edge case 👇

For a function that returns the maximum of two numbers (the larger one), which of these is an edge case?

A.

Inputs are 10 and 3, function returns 10.

B.

Inputs are -5 and -2, function returns -2.

C.

Inputs are 7 and 7, function returns 7.

D.

Inputs are 2.5 and 9.1, function returns 9.1.

Nice pick, Ron 🎯

Inputs are 7 and 7, function returns 7 is an edge case because both numbers are equal —

it’s not the “typical” larger vs smaller situation, but the function still has to behave correctly.

You’ve now:

Used the four pillars (inputs, outputs, constraints, edge cases)
Applied them to three different features (snack picker, is-even, password strength, max of two numbers)
Correctly identified each pillar in MCQs

That’s the mastery target for this lesson ☑

Before we wrap this lesson officially:

Do you want to do one more practice spec together, or are you ready to move on to the next lesson?

I'm ready to move on

Nice work on this one, Ron 🎯

You:

Took plain-English requests and turned them into four□pillar specs
Clearly named inputs, outputs, constraints, and edge cases
Showed you can do this across different problems (numbers, passwords, snacks)
This “think before typing” habit is a huge skill for real-world software work — it saves time, reduces bugs, and makes teamwork smoother.

When you’re ready, go ahead and open the next lesson in the Learn tab, and we’ll build on this.